

Project: TranspoLink - Smart Transportation & Traffic Management System

1. Introduction

TranspoLink is a web-based platform designed for municipalities, traffic departments, and public transport agencies to manage urban mobility, traffic flows, and public transport operations. It enables citizens to access traffic updates, transport schedules, and incident alerts, while administrators and officers can monitor traffic patterns, manage incidents, and ensure compliance with transport policies.

The system supports workflows for traffic monitoring, incident reporting, public transport scheduling, compliance tracking, and analytics. It ensures transparency and accountability by maintaining audit trails, dashboards, and performance metrics across transportation operations.

Actors / Users

- **Citizen:** Accesses traffic updates, transport schedules, and incident alerts.
- **Traffic Officer:** Manages traffic incidents, updates road conditions, enforces compliance.
- **Transport Operator:** Manages bus/train schedules, monitors fleet operations.
- **City Administrator:** Oversees transport operations, compliance, and reporting.
- **Compliance Officer:** Ensures adherence to transport policies, audits records.

2. Module Overview

- 2.1 Identity & Access Management
- 2.2 Traffic Incident Reporting & Management
- 2.3 Road & Traffic Flow Monitoring
- 2.4 Public Transport Scheduling & Fleet Management
- 2.5 Compliance & Audit Management
- 2.6 Reporting & Analytics
- 2.7 Notifications & Alerts

3. Architecture Overview

- **Frontend:** Angular or React for responsive dashboards.
- **Backend:** REST API-based microservices (Identity, Traffic, Transport, Compliance, Reports).
- **Database:** Relational DB (MySQL/PostgreSQL/SQL Server).
- **Deployment:** Cloud/on-prem with API gateway, WAF, centralized logging.

4. Module Wise Design

4.1 Identity & Access Management

Features:

Provides secure authentication, role-based access control, and audit logging.

Entities:

- User(UserID, Name, Role [Citizen/TrafficOfficer/TransportOperator/Admin/Compliance], Email, Phone, Status)
- AuditLog(AuditID, UserID, Action, Resource, Timestamp)

4.2 Traffic Incident Reporting & Management

Features:

Citizens and officers report traffic incidents, track resolution, and update road conditions.

Entities:

- Incident(IncidentID, ReporterID, Type [Accident/Breakdown/Roadblock], Location, Date, Status)
- Resolution(ResolutionID, IncidentID, OfficerID, Actions, Date, Status)

4.3 Road & Traffic Flow Monitoring

Features:

Monitors traffic flow data, records congestion levels, and updates road conditions.

Entities:

- RoadSegment(SegmentID, Location, Length, Status)
- TrafficFlow(FlowID, SegmentID, Volume, Speed, Date, Status)

4.4 Public Transport Scheduling & Fleet Management

Features:

Manages bus/train schedules, monitors fleet operations, tracks delays.

Entities:

- TransportRoute(RouteID, OperatorID, Type [Bus/Train], StartPoint, EndPoint, Status)
- Schedule(ScheduleID, RouteID, DepartureTime, ArrivalTime, Status)
- Fleet(FleetID, OperatorID, VehicleType, Capacity, Status)

4.5 Compliance & Audit Management

Features:

Ensures adherence to transport policies, audits records.

Entities:

- ComplianceRecord(ComplianceID, EntityID, Type [Incident/Transport], Result, Date, Notes)
- Audit(AuditID, OfficerID, Scope, Findings, Date, Status)

4.6 Reporting & Analytics

Features:

Generates dashboards and reports on traffic incidents, transport schedules, compliance, and congestion metrics.

Entities:

- Report(ReportID, Scope [Incident/Transport/Traffic], Metrics, GeneratedDate)

4.7 Notifications & Alerts

Features:

Sends alerts for traffic incidents, transport delays, compliance deadlines.

Entities:

- Notification(NotificationID, UserID, EntityID, Message, Category [Incident/Transport/Compliance], Status, CreatedDate)

5. Deployment Strategy

- **Local:** Angular/React frontend, Spring Boot/.NET Core backend, local DB.
- **Production:** Cloud/on-prem deployment, API gateway, WAF, centralized logging.

6. Database Design

Tables:

- User(UserID, Name, Role, Email, Phone, Status)
- AuditLog(AuditID, UserID, Action, Resource, Timestamp)
- Incident(IncidentID, ReporterID, Type, Location, Date, Status)
- Resolution(ResolutionID, IncidentID, OfficerID, Actions, Date, Status)
- RoadSegment(SegmentID, Location, Length, Status)
- TrafficFlow(FlowID, SegmentID, Volume, Speed, Date, Status)

- TransportRoute(RouteID, OperatorID, Type, StartPoint, EndPoint, Status)
- Schedule(ScheduleID, RouteID, DepartureTime, ArrivalTime, Status)
- Fleet(FleetID, OperatorID, VehicleType, Capacity, Status)
- ComplianceRecord(ComplianceID, EntityID, Type, Result, Date, Notes)
- Audit(AuditID, OfficerID, Scope, Findings, Date, Status)
- Report(ReportID, Scope, Metrics, GeneratedDate)
- Notification(NotificationID, UserID, EntityID, Message, Category, Status, CreatedDate)

7. User Interface Design

- **Citizen Portal:** Traffic updates, incident reporting, transport schedules.
- **Traffic Officer Dashboard:** Incident management, road monitoring, compliance tracking.
- **Transport Operator Console:** Route scheduling, fleet management, delay tracking.
- **City Administrator Panel:** Transport metrics, compliance monitoring, reporting.
- **Compliance Officer Console:** Policy monitoring, audits, compliance reporting.

8. Non-Functional Requirements

- **Performance:** Handle 100,000 concurrent users across city networks.
- **Security:** Role-based access, encrypted data storage, immutable audit logs.
- **Scalability:** Support multi-city, multi-agency rollout.
- **Availability:** 99.9% uptime.
- **Maintainability:** Modular microservices, API versioning, automated migrations.
- **Observability:** Centralized logging, KPIs (incident response time, transport punctuality, compliance adherence).

9. Assumptions & Constraints

- Initial rollout for a single city before multi-city expansion.
- Notifications limited to in-app and SMS/email alerts.
- Fully implementable using Java Spring Boot/ASP.NET Core, Angular/React, and relational DB.